# Thermal Reasoning Substrate: A Heat-Driven Shared Memory Architecture for Parallel LLM Inference

Arko Sanyal

Categories: cs.AI, cs.CL

Keywords: parallel inference, shared memory, spreading activation, multi-agent reasoning, edge deployment, personalized AI

---

## Abstract

Multi-step reasoning with large language models (LLMs) is increasingly structured as parallel branch execution, where subquestions or subtasks run concurrently and their outputs are merged before a final answer is synthesized. The dominant synchronization model for such pipelines is hard causal blocking: a downstream branch waits until all upstream branches have fully terminated before it may proceed. This serializes latency back toward sequential worst-case and wastes the computational overlap that parallelism is meant to provide.

We introduce the **Thermal Reasoning Substrate (TRS)**, a shared compact memory architecture that replaces hard causal synchronization with soft, latency-gated dependency. TRS maintains a 128-dimensional int8-quantized entity heat map — approximately 150 KB of working memory — that all parallel branches continuously read from and write to during inference. Salience is propagated across semantic neighbors via spreading activation with configurable decay over two hops. A per-entity-type z-score gate suppresses premature forwarding of low-confidence activations. A typed value bus provides exact key-value slots for numeric and precise dependencies, supporting mid-stream early forwarding: a branch may commit a value before it terminates, unblocking dependents without requiring branch completion. A personalized profile layer acts as a value predictor, reducing stall further as the profile matures over sessions.

We validate TRS on an internal SLP benchmark of 50 queries over 291 replayed transcript turns. Stream-replay calibration raises MRR from a 0.135 cold-start baseline to 0.940, exceeding a full embedding-scan engine (MRR 0.739) by a factor of 1.27, while maintaining a fixed 150 KB working set versus the engine's O(n) memory footprint. We discuss five acknowledged limitations, propose a rigorous validation experiment on MuSiQue and HotpotQA, and situate TRS in the traditions of Global Workspace Theory, spreading activation, CPU value prediction, and the Hearsay-II blackboard architecture.

---

## 1. Introduction

The last two years have witnessed a proliferation of parallel multi-agent and multi-branch LLM pipelines. Chain-of-thought reasoning can be decomposed into parallel subquestions. Tool-augmented agents spawn parallel retrieval, code execution, and summarization branches. Speculative decoding drafts multiple token sequences simultaneously. In each case, the

fundamental computational unit is no longer a single sequential model call but a directed acyclic graph of concurrent inference steps whose outputs must be combined.

Yet the synchronization mechanisms connecting these branches have lagged far behind the sophistication of the individual branches themselves. The overwhelming convention remains **hard causal blocking**: branch B, which depends on knowledge produced by branch A, suspends execution and polls until branch A signals termination. This is operationally simple and guarantees correctness — but at a steep cost. Wall-clock latency for a two-branch pipeline with one dependency collapses from the theoretical maximum of `max(lat_A, lat_B)` back toward `lat_A + lat_B` in the worst case. For deep pipelines, the regression is total: parallelism exists in code but not in time.

The root cause is a categorical mismatch between dependency granularity and synchronization granularity. Branch B's dependency on branch A is typically not a dependency on branch A's entire output. It is a dependency on one or a handful of **entities** — a named quantity, a resolved reference, a committed intermediate conclusion — that branch A will produce at some point during its inference, often long before that inference concludes. Hard blocking waits for a binary signal that destroys this distinction.

We propose that reasoning dependencies should be classified at entity granularity into two types: **informational dependencies**, which require salience signals about what topics are active and relevant across branches, and **value dependencies**, which require a specific committed quantity. Informational dependencies are appropriately served by a continuous low-bandwidth broadcast medium that propagates semantic heat. Value dependencies are appropriately served by a typed slot that is committed the moment confidence crosses a threshold, not the moment the producing branch terminates.

The **Thermal Reasoning Substrate (TRS)** is an architecture that provides exactly these two channels, together with a third: a personalized behavioral profile that functions as a value predictor, reducing value-bus stall as it matures. The informational bus is a 128-dimensional int8-quantized entity heat map with spreading activation and z-score gating. The value bus is a typed key-value store with mid-stream commit semantics. The profile layer accumulates user behavioral signal to improve prediction accuracy over time.

TRS is designed to be deployable at the edge. The 150 KB working set makes it compatible with 1-3B parameter models on consumer hardware, targeting domain-specific tasks where a 7B model with flat context would require substantially more memory. The quantized heat map is the primary data structure; no dense embedding index is required at runtime.

The remainder of this paper is organized as follows. Section 2 surveys the theoretical and systems predecessors that motivate TRS. Section 3 describes the architecture in detail. Section 4 provides a theoretical analysis of the latency model and dependency classification. Section 5 describes the implementation components. Section 6 presents experimental results on the SLP benchmark. Section 7 acknowledges limitations. Section 8 outlines future work including a rigorous experimental protocol. Section 9 situates TRS as model-agnostic memory middleware applicable beyond the parallel inference setting. Section 10 concludes. Appendix A discusses population-scale emergence and socioeconomic stratification in aggregated heat

map profiles.

---

## 2. Background and Related Work

### 2.1 Global Workspace Theory

The most direct theoretical antecedent of TRS is Global Workspace Theory (GWT), first articulated by Baars [1] and extended with neural grounding by Dehaene and colleagues [2]. GWT proposes that cognition is organized as a set of specialized, largely unconscious processing modules that compete for access to a shared global workspace. Information that wins this competition is broadcast widely, becoming available to all modules simultaneously. Conscious perception, in this account, is the event of information entering the global workspace and being broadcast.

For our purposes, the architectural consequence is more important than the phenomenological claim: a shared broadcast medium with competitive admission achieves low-latency information sharing across otherwise parallel specialized processors without requiring those processors to maintain bilateral communication channels. The global workspace is a scalability device. GWT predicts that workspace capacity is limited — consistent with the well-studied limits of working memory — and that access is gated by a relevance or salience signal. Both predictions are reproduced in TRS's design: the 150 KB working set is a hard capacity limit, and z-score gating implements a relevance admission criterion.

### 2.2 Spreading Activation

Collins and Loftus [3] introduced spreading activation as a model of semantic memory retrieval. In their model, a concept node activated by a stimulus sends activation energy to semantically related nodes, with energy decaying as a function of semantic distance. The resulting pattern of activation represents the current focus of semantic attention across the conceptual network.

TRS borrows this mechanism directly for the informational bus. When a branch writes a heat increment to an entity, the heat map propagates that increment to the entity's semantic neighbors up to two hops, attenuated by empirically tuned decay coefficients. The effect is that a branch reasoning about "battery capacity" implicitly activates neighboring entities such as "charging speed" and "energy density" even before any branch has explicitly mentioned them. When a downstream branch queries the heat map, it encounters a pre-activated semantic neighborhood, replicating the priming effect that makes human semantic memory fast and coherent.

The two-hop limit on propagation is a deliberate design choice. Unbounded propagation risks diffusing salience too broadly, diluting the signal until it conveys little information about actual reasoning focus. A single hop limits priming to immediate conceptual neighbors; two hops allows the activation of likely bridge entities — the intermediate concepts that connect two distant topics in multi-hop reasoning — without activating the full semantic neighborhood of the second-order neighbors. This range reflects a trade-off between priming breadth and signal specificity that we expect to be task-dependent; future work should investigate whether adaptive hop counts improve performance on specific benchmark categories.

### 2.3 CPU Out-of-Order Execution and Value Prediction

The Tomasulo algorithm [4], originally implemented in the IBM System/360 Model 91, introduced the concept of register renaming and a common data bus to allow instructions to execute out of order while preserving apparent sequential semantics. The central insight is that a true data dependency need not impose sequential execution order; it imposes only a data availability constraint. Tomasulo's reservation stations allow an instruction to issue and begin execution while waiting for a prior result to appear on the common data bus, rather than blocking the entire pipeline.

TRS's value bus is a direct application of this principle at the level of reasoning branches. A branch that needs a value from another branch does not need to wait for that branch to terminate; it needs to wait for that value to be committed to the value bus. If the producing branch can commit early — which is possible whenever the value is determined before the full reasoning chain concludes — the dependent branch resumes without waiting for the remainder of the producing branch's output.

Value prediction [5], introduced by Lipasti and colleagues, extends this further: a predictor circuit estimates the value that an instruction will produce before the instruction executes, allowing dependent instructions to proceed speculatively. If the prediction is correct, no rollback is needed. TRS's profile layer implements an analogous mechanism: as the profile matures, it predicts the values that forthcoming branches will commit, allowing dependents to proceed with predicted values and experience zero stall in the high-accuracy regime. The analogy is imperfect — branch predictions are richer semantic objects than instruction output values — but the architectural logic is identical.

### 2.4 The Hearsay-II Blackboard Architecture

Erman and colleagues [6] described Hearsay-II, a speech understanding system organized around a shared blackboard data structure on which multiple knowledge sources post hypotheses at different levels of abstraction — phonemes, syllables, words, phrases. Any knowledge source may read the blackboard at any time and post new hypotheses triggered by the current blackboard state. There is no direct communication between knowledge sources; all coordination is mediated through the blackboard.

TRS differs from Hearsay-II in several important respects. Hearsay-II operates in discrete rounds with a scheduler selecting which knowledge source to activate; TRS is continuous, with branches writing and reading the heat map on every token or sentence boundary. Hearsay-II's hypotheses are symbolic structures; TRS's heat map is a quantized vector representation optimized for memory footprint and read throughput rather than symbolic interpretability. But the fundamental architectural intuition — that shared typed state enables emergent coordination among otherwise independent processors — is the same, and the blackboard's typed hypothesis levels correspond loosely to TRS's tiered context cascade.

### 2.5 Prior Work on Parallel LLM Inference

Several recent systems have addressed the problem of parallel LLM branch coordination. BIGMAS [7] introduces a shared workspace for multi-agent systems but uses a text-based representation and imposes hard synchronization rounds between agents. The text format

incurs substantial token overhead; the round synchronization collapses latency back toward sequential execution. Theater of Mind [8] draws on GWT to construct a prompt-sharing mechanism for parallel agents but likewise relies on text prompts as the communication medium, foregoing the quantized vector representation that makes continuous low-overhead broadcast feasible.

Parallel-Synthesis [9] addresses the KV cache prefill problem in parallel decoding, optimizing the case where multiple branches share a prefix. It achieves significant prefill savings but the branches' outputs remain independent until branch completion; there is no entity-level dynamic exchange during inference. Native Parallel Reasoner [10] schedules reasoning steps using topological sort derived from a dependency graph, enabling some parallelism but retaining hard topological blocking rather than soft threshold gating. Blackboard LLM [11] is architecturally closest to TRS — it implements a shared text store through which agents coordinate — but uses discrete synchronous rounds and does not implement heat propagation or spreading activation.

Speculative decoding approaches, including SpecBranch [12], operate at the token level rather than the entity level. A draft model generates token sequences that a verifier model accepts or rejects. This is a powerful throughput optimization but addresses a different problem than TRS: SpecBranch's branches are token-level hypotheses for a single generation, not reasoning branches that produce semantically distinct intermediate conclusions that must be coordinated across an inference graph.

The gap that TRS fills is the combination of: (a) a continuous, quantized, sub-token-overhead communication medium; (b) soft threshold-gated rather than hard blocked dependency resolution; (c) spreading activation for semantic priming across branches; and (d) a typed value bus with mid-stream commit semantics. No prior system implements all four simultaneously.

---

## 3. Architecture

TRS consists of three channels that together replace the hard synchronization boundary between parallel reasoning branches: the informational bus, the value bus, and the profile layer. We describe each in turn, followed by the stream extraction and context cascade mechanisms that drive them.

### 3.1 Informational Bus

The informational bus is a shared entity heat map. Each entity recognized in the discourse — named entities, domain concepts, relationship types, task-relevant quantities — occupies a 128-dimensional int8-quantized vector slot. At 1 byte per dimension plus metadata overhead, each slot occupies approximately 322 bytes. The working set at full capacity accommodates approximately 465 simultaneous active entities within 150 KB.

When a branch emits a token or sentence, a lightweight stream extractor identifies mentioned entities and increments their heat values. Heat is represented as a scalar attached to each entity's vector slot; the 128-dimensional representation encodes semantic identity while the heat scalar encodes current salience. Heat accumulates multiplicatively: repeated mention

within a short context window raises heat faster than infrequent mention. Between writes, heat decays exponentially at an empirically tuned rate, so entities that recede from the discourse cool passively without requiring explicit housekeeping.

Spreading activation operates on every write event. When entity E receives a heat increment, the system identifies E's semantic neighbors within two hops of the entity graph and distributes a fraction of the increment to each, attenuated by empirically tuned distance-dependent decay coefficients. The entity graph is precomputed for the deployment domain and loaded at system initialization. First-hop neighbors receive a larger fraction; second-hop neighbors receive a substantially smaller fraction. The result is that every explicit mention propagates implicit salience to the conceptual neighborhood, enabling cross-branch priming without any explicit inter-branch message passing.

A z-score gate prevents premature forwarding of low-confidence activations. For each entity type — person, location, numeric quantity, domain concept, and others — the system maintains a running mean and variance of heat values observed during the current session. An entity's heat value is forwarded to dependent branches only when its z-score exceeds an empirically tuned threshold for that entity type. This suppresses noise — entities that appear once in passing — from triggering false dependencies. The z-score statistics are updated continuously, adapting to the session's entity distribution.

Reads from the informational bus return a ranked list of entities above the z-score gate threshold, together with their 128-dimensional representations. A branch that reads the bus encounters not only the entities it has itself activated, but also entities activated by sibling branches via spreading activation — achieving semantic priming without any explicit inter-branch message passing.

### 3.2 Value Bus

The value bus is a typed key-value store for exact, precise dependencies. Keys are entity-attribute pairs. Values are typed: integer, float, string, boolean, or structured. Each slot carries a confidence score and a committed flag.

The critical semantic property of the value bus is **mid-stream commit**: a producing branch may write a value to the bus and set the committed flag before the branch terminates, as soon as confidence in that value crosses an empirically tuned threshold. A branch that concludes "the answer to subquestion Q2 is 47" with high confidence at sentence 3 of a 20-sentence response may commit that answer immediately. Any dependent branch that has been waiting on this slot is unblocked at that point, not at the end of the 20-sentence response.

This converts value-bus blocking from "wait until producing branch terminates" to "wait until producing branch commits value," which in the best case is nearly simultaneous with the value being determined during reasoning. In the worst case — where a branch's answer is not determined until the final sentence — stall duration equals hard blocking. But across a diverse mix of reasoning tasks, value commit is expected to occur substantially before branch termination on average, because conclusions are typically drawn before supporting elaboration is complete.

Conflict resolution is needed when two branches write to the same value-bus slot simultaneously. TRS uses a last-write-wins policy on the committed flag: the most recently committed value is the one that dependents read. This is safe under the assumption that the producing branches are exploring compatible reasoning paths; divergent paths that produce conflicting values signal a reasoning inconsistency that should be surfaced to the orchestrating agent rather than resolved silently. Conflict detection triggers a flag that the orchestrator may act on.

Before the profile layer matures, the value bus supports **tentative writes**: a branch may post a predicted value with confidence below the commit threshold and a tentative flag set. Dependents that choose to proceed on tentative values do so speculatively; rollback machinery restores their state if the tentative value is later superseded by a committed value that differs. The rollback machinery adds implementation complexity, which is why tentative writes are disabled until sufficient profile data is available to bound the probability of misprediction.

### 3.3 Profile Layer

The profile layer is a persistent, user-specific behavioral model that accumulates signal across sessions. It is not a monolithic embedding of a user's history; it is a structured prediction model for value-bus slots. For each recurring entity-attribute pair — quantities the user frequently asks about, preferences the user has expressed, facts the user has confirmed — the profile stores a distribution over likely values together with a staleness timestamp and a prediction accuracy tracker.

At inference time, the profile layer is consulted before the value bus is queried. If the profile contains a high-confidence prediction for a needed slot, the dependent branch may proceed immediately with that predicted value rather than waiting for any producing branch to commit. The prediction is treated as a tentative value with confidence equal to the profile's historical accuracy on that slot type.

Profile maturity is measured by prediction accuracy on historical value-bus slots. A freshly initialized profile has no predictions and contributes no stall reduction. As sessions accumulate, the profile learns which entity-attribute pairs are predictable for this user and which are not. High-variance quantities are flagged as unpredictable and excluded from prediction. Low-variance quantities that recur frequently are the profile's primary contribution to stall reduction. The profile thus produces increasing returns: early sessions derive no benefit from the profile layer; mature profiles substantially eliminate value-bus stall for common query types.

A staleness detector monitors prediction accuracy on a rolling window. When accuracy drops below a configurable threshold — indicating that the user's behavioral patterns have shifted — the profile is flagged for recalibration and confidence is reduced until accuracy recovers. This prevents the compounding failure mode where a stale profile provides high-confidence wrong predictions that systematically mislead dependent branches.

### 3.4 Stream Extraction

A key design choice in TRS is that heat map updates occur on intermediate tokens and sentences during branch inference, not on complete branch outputs. A component called the

**stream watcher** subscribes to the output stream of each branch and fires a `process_turn()` callback on every output unit — configurable as a token, a sentence boundary, or a sliding window of tokens.

This makes TRS's heat map a live representation of the current inferential state of all branches, not a post-hoc summary of completed branches. A branch that begins its reasoning with "The key issue here is energy storage..." immediately activates energy-storage entities for sibling branches, even while the branch is still generating its first paragraph. By the time a dependent branch needs to know what the first branch considers important, the heat map has been warming for the entire duration of the first branch's generation.

The stream watcher is lightweight by design. It operates on text, not embeddings; entity extraction uses a domain-specific gazetteer or a compact NER model appropriate to the deployment context. At edge deployment scales, the stream watcher's extraction overhead is budgeted to be negligible relative to the main model's generation time.

### 3.5 Three-Tier Context Cascade

The informational bus is complemented by a three-tier context cascade that governs how much context about a hot entity is materialized for a querying branch.

The **HOT tier** (approximately 32 bytes per entity) contains the headline representation: entity name, type, canonical identifier, and current heat scalar. This tier is always loaded in working memory for all entities above the z-score gate threshold. Reading the HOT tier costs a memory lookup; no I/O is required.

The **MILD tier** (approximately 160 bytes per entity) contains a semantic neighbor preview: the top-k neighbors in the entity graph, their heat values, and a short textual description. The MILD tier is loaded on demand for entities whose spreading-activation neighborhood is being queried, supporting context-aware queries without materializing the full entity record.

The **COLD tier** (approximately 512 bytes per entity) contains the expanded entity record: full textual description, historical value-bus entries, profile predictions, and provenance tags indicating which branches have contributed to the entity's current heat. Only the top-3 entities by heat are eligible for COLD tier materialization, and only when a branch explicitly requests full context. This prevents the cold tier from functioning as a de facto full embedding scan.

The cascade is designed so that the vast majority of informational bus reads complete at the HOT tier. Only deliberate deep-context queries pay the COLD tier cost. The three-tier structure keeps the effective working memory footprint close to the 150 KB headline figure even when cold-tier data is present.

### 3.6 Out-of-Corpus Pre-Gate

Before any branch attempts to read or write the heat map, a pre-gate classifier determines whether the query belongs to the domain for which the entity graph was constructed. Queries classified as out-of-corpus (OOC) are flagged before any embedding computation occurs. This prevents OOC queries from polluting the entity heat map with activations in the wrong semantic neighborhood, and allows the orchestrator to route them to a fallback mechanism without

wasting the latency of a failed heat-map lookup.

The OOC classifier is a lightweight intent classifier trained on positive (in-corpus) and negative (OOC) query examples for the deployment domain. It is the first computation performed on any incoming query, taking precedence over stream watcher activation.

A **dynamic $\tau$ gate** adjusts the OOC threshold based on two signals: **temporal coherence** (whether the current query continues a recent discourse thread, measured by heat-map similarity between the current query's entity set and the recent hot entities) and **era density** (whether the domain entity graph contains sufficient coverage for the query's time period or domain region, measured by graph density in the relevant semantic neighborhood). A query that is temporally coherent with the recent discourse but falls in a low-density region of the graph is treated as potentially OOC even if its surface form looks in-domain. The dynamic $\tau$ gate is tuned to prefer false positives in low-density regions, because the cost of a low-quality in-domain response exceeds the cost of a slightly slower fallback-routed response.

---

## 4. Theoretical Analysis

### 4.1 Latency Model

Let a parallel reasoning pipeline consist of N branches with a partial order of value dependencies. Under hard blocking, the wall-clock latency of the pipeline is determined by the critical path through the dependency graph, where each edge represents a full branch termination wait. If branch A terminates at time $t_A$ and branch B depends on A's output, B's start time is no earlier than $t_A$, and the pipeline latency for the A to B edge is at least $t_A + t_B$.

Under TRS, the wait on each dependency edge is replaced by a wait for a value commit or a heat threshold crossing. Let $c_A$ denote the time at which branch A commits its value to the value bus. If $c_A < t_A$ (early commit), the pipeline latency for the A-to-B dependency edge becomes $c_A + t_B < t_A + t_B$. The savings per dependency edge is $t_A - c_A$, the duration of branch A's post-commit generation. Across a pipeline with K dependency edges, total savings are the sum over all edges of the post-commit generation duration.

For informational bus dependencies — where B does not require a precise value from A but merely needs to know what topics A considers relevant — the threshold crossing occurs even earlier, typically within the first few sentences of A's generation. In this case the latency savings approach $t_A$: the informational dependency contributes negligible latency to the critical path.

The profile layer introduces a third regime: predicted value dependencies, where B does not wait for A at all because the profile supplies a prediction with sufficient confidence. In this regime the dependency edge contributes zero latency to the critical path. The prevalence of this regime increases with profile maturity, producing the compounding latency reduction described in Section 4.3.

### 4.2 Dependency Classification

We define a dependency taxonomy with three classes:

**Informational dependencies** arise when branch B would benefit from knowing that branch A considers entity E salient, but B does not need to know A's specific value for any attribute of E. These dependencies are fully served by the informational bus. They impose no stall: B reads the heat map and proceeds, incorporating the spreading activation signal.

**Value dependencies** arise when branch B needs branch A's answer to a specific question. These impose stall equal to the time between B's need for the value and A's commit of that value to the value bus. Value dependencies are subdivided into **early-commit** (A determines the answer early in its reasoning and commits promptly) and **late-commit** (A's answer emerges only near the end of its generation). Early-commit dependencies are the primary target of TRS's latency reduction; late-commit dependencies achieve savings only from the profile layer.

**Structural dependencies** arise when B needs A's entire output — for example, when B is a synthesis branch whose input is the concatenated outputs of all parallel reasoning branches. Structural dependencies cannot be served by TRS; they require full branch completion. TRS does not attempt to reduce latency for structural dependencies.

In practice, the distribution of dependency types varies substantially by task. Multi-hop factoid reasoning involves many value dependencies that we expect to be predominantly early-commit: the bridge entity is typically identified early in the reasoning chain. Open-ended generation tasks have fewer precise value dependencies and more structural dependencies. TRS's gains are largest in the former category.

### 4.3 Profile Maturity Curve

The profile layer's contribution to stall reduction follows a maturity curve. Let $p_t$ denote the prediction accuracy on value-bus slots at session $t$. The expected stall reduction from the profile layer is proportional to $p_t$ times the fraction of value dependencies that are profile-predictable — those that recur with low variance across sessions.

For a new user ($t = 0$), $p_0 = 0$ and the profile contributes nothing. As sessions accumulate, $p_t$ increases toward an asymptote determined by the inherent predictability of the user's query patterns in the deployed domain. In a domain-specific assistant used by the same person daily for a narrow task, we expect the asymptote to be high: the user asks about the same entities, the same procedural questions, the same reference values. In a general-purpose assistant with high query diversity, the asymptote is lower.

The staleness detector introduces a discontinuity in $p_t$ when behavioral patterns shift: the detected shift resets confidence, causing a temporary regression before the profile re-adapts. The staleness window is an empirically tuned hyperparameter that trades responsiveness to behavioral change against stability of predictions.

---

## 5. Implementation

### 5.1 SLP Store

The SLP (Salience-Latency Profile) store is the implementation of TRS's informational bus and value bus as a unified in-process data structure. It is implemented as a fixed-size array of entity records in shared memory, with a hash index on entity identifier for O(1) lookup. The fixed-size constraint enforces the 150 KB working set budget.

Entity records contain: a 128-dimensional int8 vector, a float32 heat scalar, a uint32 write timestamp, a uint8 entity type code, and a 32-byte identifier field. The total per-record size is 322 bytes including alignment padding. At 150 KB, the store accommodates 465 records. When the store is full, a heat-ranked least-recently-used (LRU) policy evicts the coldest entity to make room for a new one. This ensures that the active conceptual frontier is always present, at the cost of potentially evicting entities that have temporarily cooled but may regain relevance.

Value bus slots are stored separately from entity heat records to avoid false sharing in concurrent writes. Each slot is a fixed-size structure: a 32-byte key (entity ID plus attribute name hash), a 64-byte value field accommodating all scalar types and small strings, a float32 confidence score, a uint8 committed flag, and a uint8 tentative flag. Value bus reads and writes are synchronized with a lightweight spinlock per slot, chosen over a mutex to minimize overhead for the expected common case of non-contended access.

### 5.2 Stream Watcher

The stream watcher subscribes to each branch's output stream and calls `process_turn()` on each unit. `process_turn()` runs entity extraction on the text unit, looks up each identified entity in the SLP store, and applies the heat increment together with spreading activation.

Entity extraction at stream-watch latency must be fast. The implementation uses a domain-specific gazetteer for high-precision entity identification, supplemented by a lightweight NER model for entities not in the gazetteer. The gazetteer is constructed offline from the entity graph used for spreading activation, ensuring that all gazetteer entities have precomputed neighbor sets.

Heat increment magnitude is a function of mention frequency in the current context window, entity type, and emphasis markers in the text such as capitalization or explicit annotation. These weights are empirically tuned per deployment domain.

### 5.3 Label Engine

The label engine is responsible for entity type classification and z-score gate computation. It maintains per-type running statistics and computes z-scores on read requests. Label assignment uses the entity type code stored in the SLP record, which is set at the time the entity is first inserted into the store.

The z-score threshold per entity type is an empirically tuned hyperparameter. In the SLP benchmark deployment, separate thresholds were tuned for infrastructure concepts, task-management entities, and adversarial probe entities based on observed heat distributions in the training transcript corpus. This per-type tuning is important: numeric quantities exhibit different heat distributions than named entities, and a uniform threshold would either over-forward numeric noise or under-forward genuine named entity salience.

### 5.4 Dynamic $\tau$ Gate and OOC Classification

The OOC pre-gate uses a trained binary classifier that inspects the incoming query and returns an OOC probability. If this probability exceeds the threshold $\tau$, the query is routed to the fallback path before any SLP store access occurs.

The dynamic $\tau$ gate adjusts this threshold based on temporal coherence and era density signals as described in Section 3.6. The gate prevents two failure modes: false negatives (OOC queries that slip through because they share vocabulary with in-domain queries) and false positives (in-domain queries that appear novel because the user has shifted to a less-covered subdomain). The $\tau$ value, its adjustment bounds, and the sensitivity to each signal are empirically tuned hyperparameters specific to the deployment domain and expected query distribution.

### 5.5 Production Deployment: Forge as First TRS Agent

Forge is an autonomous AI coding and research agent deployed on the Billo AI platform (billo.tech). It constitutes the first production deployment of the TRS architecture outside of the benchmark setting.

Forge uses the LOCI context engine as its SLP store, namespaced under `user_id=forge`. Before each task begins, Forge queries `/v1/context/query` to retrieve related past task completions, providing a pre-warmed informational bus seeded with entities from prior executions. After each task, Forge commits metadata to the entity store as records with `entity_type=forge_completion`, feeding the next session's retrieval context. This pre-task query / post-task commit cycle is precisely the stream-watcher and SLP-write cycle described in Section 5.2, instantiated at task granularity rather than token granularity.

Forge task types span all three categories evaluated in Section 6.1. Infrastructure tasks correspond to infra queries (port checks, service restarts, shell command execution). Code patch tasks correspond to task-management queries (file edits, diff generation, patch application). Novel generation tasks — writing new tools, generating HTML pages, research synthesis — correspond to the adversarial category, in that they present problems with no direct antecedent in the warm entity set.

The deployment uses a `forge_capability_journal` table to log per-task outcomes, accumulating a train/test-splittable dataset along natural session boundaries. Task retrieval MRR on the live `user_id=forge` query log constitutes an in-production replication of the SLP-50 benchmark. As of this writing the deployment has fewer than 200 recorded task completions, below the threshold for a statistically meaningful category breakdown. We anticipate reporting complete production MRR in a subsequent revision once the journal reaches 200+ completions.

---

## 6. Experiments

### 6.1 Experimental Setup

**Corpus collection.** The SLP-50 benchmark is grounded in a corpus of 291 transcript turns drawn from consecutive operational sessions of the Billo AI platform conducted over a 72-hour

window (2026-06-30 to 2026-07-02). Sessions were not curated or filtered: every turn from the raw session logs was included, resulting in a naturalistic distribution across task types. The corpus spans three broad categories of operational work: (1) infrastructure management — service debugging, port configuration, process management, SSH and tunnel operations (142 turns); (2) code patch tasks — file editing, patch application, function refactoring, diff generation (89 turns); (3) research and synthesis tasks — literature review, competitive analysis, benchmark design, report drafting (60 turns). All turns were generated by a single user (arko.sanyal@gmail.com) interacting with Claude Sonnet 4.6 as the Forge agent.

**Query construction.** From the 291-turn corpus, 50 queries were manually constructed by the author to reflect retrieval requests that would plausibly arise during live operation. Queries were not randomly sampled from the corpus; they were stratified by category to ensure coverage across all three query types. For each query, the target entity or value was the one that appeared most prominently in the relevant portion of the corpus — the entity a dependent branch would need to locate to correctly complete the query's implicit reasoning step. Query surface forms were written to be natural-language questions (e.g., "What port is the broker running on?", "Which model is currently loaded for Forge?") rather than keyword lookups, to test retrieval under realistic phrasing variation.

**Ground truth labeling.** For each query, the correct answer entity was labeled by the author by inspecting the corresponding session transcript segment and identifying the entity whose heat should be highest given the query. Labels were assigned at entity-type resolution: for infrastructure queries, the label is the entity record matching the queried parameter (e.g., the `broker_port` entity with value 11436); for task queries, the label is the most recent task-related entity (e.g., a `forge_completion` entity for a Forge task query); for adversarial queries, the expected behavior is OOC rejection and there is no correct entity, so MRR is measured on whether the top-returned entity is correctly suppressed (rank 1 = OOC gate triggered). A second labeling pass was performed on a 20-query random subset to verify consistency; all 20 labels agreed with the first pass, giving 100% inter-pass agreement.

**Evaluation metric.** We use Mean Reciprocal Rank (MRR) of the ground-truth entity in the ranked output of the SLP store. MRR is appropriate here because TRS returns a ranked list from the heat map and dependent branches consume the top-ranked entity first — the time cost to a dependent branch is proportional to how far down the ranked list the correct entity appears. MRR is also appropriate as a single scalar summary because the number of queries per category is small enough that macro-averaged metrics by category would have high variance; per-category results are shown in Table 2.

We evaluate TRS on this benchmark dataset, denoted SLP-50, consisting of 50 queries drawn from three categories: infrastructure reasoning (infra, 15 queries), task-management reasoning (task, 21 queries), and adversarial probes designed to test the OOC gate and z-score robustness (adversarial, 14 queries).

We compare three conditions:

**Cold start:** the SLP store is empty; all queries are answered from the cold store. This establishes the baseline for a system with no prior session history and quantifies the gap that

stream-replay warming closes.

**TRS (stream replay):** the 291 transcript turns are replayed through the stream watcher before benchmark evaluation, warming the heat map. This simulates a system that has been operational for one or more sessions.

**Full engine:** a full embedding-scan retrieval engine that embeds all 291 transcript turns and retrieves by cosine similarity at query time. This represents the standard retrieval-augmented generation approach with O(n) memory footprint and query-time embedding cost.

### 6.2 Results

Table 1 summarizes MRR results across conditions.

| Condition | MRR | Working Set |
|---|---|---|
| Cold start (TRS) | 0.135 | 150 KB |
| TRS (stream replay) | **0.940** | 150 KB |
| Full engine | 0.739 | O(n) |

The cold-start MRR of 0.135 reflects the expected behavior of an empty heat map: without prior session signal, the informational bus has no warm entities, and the store returns results that are essentially unranked for queries whose entities are not in the initialization set. This baseline confirms that TRS's gains are attributable to the stream-replay warming mechanism, not to any prior knowledge embedded in the system at initialization.

After replaying 291 turns through the stream watcher, TRS achieves MRR 0.940, substantially exceeding both the cold-start baseline and the full engine. The TRS advantage over the full engine reflects the benefit of heat-weighted retrieval over cosine similarity: heat captures recency and frequency of mention in addition to semantic similarity, which is more relevant for multi-turn reasoning contexts where recently and frequently discussed entities are more likely to be the correct retrieval targets. Cosine similarity treats all past mentions equally regardless of recency, an inappropriate assumption for conversational systems where current context dominates.

Table 2 breaks down MRR by query category.

| Category | TRS MRR | Full Engine MRR |
|---|---|---|
| Infra | 1.000 | 0.821 |
| Task | 0.944 | 0.714 |
| Adversarial | 0.909 | 0.681 |

TRS achieves perfect MRR on infrastructure queries, reflecting the well-structured nature of infrastructure entities — ports, services, hostnames, configuration parameters — in the entity graph. Task queries show a small degradation from perfect, consistent with the higher variability of natural language task descriptions where the same underlying task may be described with different surface forms across sessions. Adversarial queries show MRR 0.909,

confirming that the z-score gate and OOC pre-gate successfully suppress most adversarial activations.

The full engine performs worse than TRS on all categories, with the largest gap on adversarial queries (0.909 vs 0.681). This is consistent with the known susceptibility of cosine similarity retrieval to lexically similar but semantically irrelevant queries: adversarial probes that share vocabulary with the corpus but refer to different entities achieve high cosine similarity and are retrieved near the top of the ranking. The z-score gate, by requiring sustained heat above a session-specific baseline, is inherently more robust to such single-mention lexical matches.

### 6.3 Graph Heat Diffusion as a Non-Autoregressive Retrieval Alternative

Motivated by the analogy between TRS spreading activation and physical heat diffusion, we conducted a preliminary investigation into Graph Heat Diffusion [Chen et al., 2018] as a retrieval substrate. Rather than accumulating heat via the entity-type z-score cascade, we model retrieval as a steady-state heat equation over the LOCI entity graph:

$$\partial\phi/\partial t = -L\phi + q$$

where L is the graph Laplacian constructed from entity co-occurrence edge weights and cosine similarity above threshold 0.25, and q is the query-seeded source term (entity cosine similarities to the query embedding, thresholded to positive values). The steady state $\phi^* = -L^{\blacksquare}q$ gives a unique solution with no spurious attractors — a theoretical advantage over Modern Hopfield Networks, whose retrieval capacity is bounded by 0.14N (where N is the number of stored patterns), causing attractor collapse at the scales relevant to Billo's entity graph.

We benchmarked three conditions on the 17-entity clean LOCI graph (post-noise-entity removal) with nomic-embed-text-v1.5 embeddings (768d, frozen):

| Method | MRR | Avg latency |
|---|---|---|
| Baseline cosine | 0.950 | <1ms |
| Modern Hopfield (β=5) | 0.164 | <1ms |
| Graph Heat ODE (Euler, 80 steps) | 0.933 | <1ms |

Hopfield collapses to a dominant attractor (MRR 0.164), confirming the spurious attractor failure mode at this graph scale. Graph Heat ODE achieves MRR 0.933, near-identical to baseline cosine (0.950), with a marginal penalty (–0.017) attributable to heat leakage through the graph from the target entity to neighbors.

**Interpretation.** The near-tie between Graph Heat ODE and baseline cosine on direct-paraphrase queries is expected: when query embeddings are semantically close to the target entity, cosine similarity already provides a strong signal, leaving little room for graph propagation to add. The theoretical advantage of Graph Heat ODE emerges on **episodic multi-hop** queries — queries that are semantically distant from the target entity but graph-connected through intermediate entities that share the user's session history. This property is not testable on the current 17-entity graph, which lacks the session depth required

to accumulate meaningful episodic edges. We expect the advantage to materialize as the LOCI graph grows beyond 200 entities with at least 3 sessions of co-occurrence history, and treat this as a core validation target for the Billo-1 G-NODE architecture described in Section 8.6.

The Hopfield result constitutes an empirical falsification of Modern Hopfield Networks as a retrieval substrate for personalized entity graphs at this scale. We include this result for completeness and to document the design decision to proceed with Graph Heat ODE and Neural ODE training rather than Hopfield networks.

### 6.4 Memory Efficiency

TRS's working set of 150 KB is fixed regardless of corpus size. The full engine's working set grows linearly with the number of transcript turns: at 291 turns with a 1536-dimensional float32 embedding, the embedding matrix alone requires approximately 1.7 MB, more than 11 times TRS's working set. At larger corpus sizes — thousands of turns across a long-running deployment — the disparity grows proportionally, while TRS's working set remains constant at 150 KB with LRU eviction.

This memory efficiency is the primary design motivation for edge deployment. A 1-3B parameter model on a device with 4-8 GB of RAM can host TRS alongside the model without significant memory pressure. The full engine at scale would compete with the model's KV cache for RAM, degrading inference throughput — a trade-off that does not exist for TRS.

---

## 7. Limitations

We identify five acknowledged limitations of TRS in its current form.

**Limitation 1: Stationarity assumption in z-score gating.** The z-score gate assumes that the distribution of heat values per entity type is approximately stationary within a session. If entity type distributions shift mid-session — for example, if a conversation transitions from a domain where numeric quantities are rare to one where they are common — the z-score thresholds become miscalibrated. An entity type that normally requires high heat to pass the gate may be suppressed or over-forwarded during a distribution shift. The principled fix is an entropy-based recalibration trigger: when the observed distribution diverges from the running baseline by more than a threshold measured by KL divergence, the running statistics are reset and re-estimated from recent observations. This recalibration introduces a brief accuracy degradation during re-estimation.

**Limitation 2: Store saturation at 465 active entities.** The 150 KB working set accommodates approximately 465 active entity records. In reasoning tasks that involve a large cast of distinct entities — extended multi-hop chains over dense knowledge graphs, for example — the store may saturate, causing LRU eviction of entities that have temporarily cooled but remain relevant. The heat-ranked LRU policy minimizes this risk for the most salient recent entities but cannot protect entities that are referenced infrequently yet critically. A two-level store — fast fixed-size primary in RAM, larger secondary on disk or extended memory — would relieve this constraint at the cost of access latency for secondary retrievals.

**Limitation 3: Rollback machinery for tentative value bus commits.** The value bus's tentative write mechanism requires that dependent branches be able to roll back their reasoning if a tentative value is superseded by a committed value that differs. Implementing this rollback machinery cleanly is non-trivial: branches must checkpoint their intermediate reasoning state at the point of reading a tentative value, and the rollback must restore that checkpoint and resume with the correct value. Until the profile is mature enough to bound misprediction probability to an acceptable level, tentative writes should be disabled, foregoing the stall reduction they provide. Early-session performance is therefore limited to hard-commit stall reduction, not speculative stall elimination.

**Limitation 4: False convergence risk from warmed entities.** A branch that reads a hot entity from the informational bus may incorporate it into its reasoning even when that entity was activated by a sibling branch's reasoning path that turns out to be incorrect. If the sibling's path is wrong and the hot entity reflects that erroneous path, the reading branch imports the error. The z-score gate reduces the probability of false convergence from ephemeral activations, but sustained incorrect activations from a consistently wrong branch may pass the gate. We identify a kill criterion for the z-score gate design: if false convergence exceeds 15% of queries on HotpotQA or MuSiQue in the recommended validation experiment (see Section 8), the z-score gate must be redesigned, potentially incorporating branch-provenance tagging that allows readers to weight activations by the inferred reliability of the writing branch.

**Limitation 5: Write contention under high branch counts.** With N branches writing to the SLP store simultaneously, write contention on entity records increases with N. The current per-record spinlock strategy is appropriate for N of 4 or fewer but may degrade for larger branch counts, particularly under bursty write patterns where all branches simultaneously mention the same entity. The merge strategy for concurrent heat increments uses exponential heat averaging: the resulting heat value is a weighted combination of the existing heat and the incoming increment rather than a simple addition. This prevents a burst of simultaneous writes from excessively inflating a single entity's heat, at the cost of slightly underweighting genuine consensus activations. For the value bus, last-write-wins semantics are simple but may cause information loss if two branches commit different yet both-valid values. Future designs should consider a more nuanced merge policy for value bus conflicts.

---

## 8. Future Work

### 8.1 Recommended Validation Experiment

The SLP-50 benchmark, while encouraging, is an internal evaluation on a specific deployment domain with a single user's session history. Rigorous validation of TRS's core claims requires a controlled experiment on a standard multi-hop reasoning benchmark.

We recommend the following experimental protocol. The benchmark should be MuSiQue or HotpotQA, both of which require multi-hop reasoning where intermediate bridge entities must be identified before the final answer can be computed — the canonical setting for value dependencies. The model should be Llama 3.2-3B, representing the edge-deployment target and ensuring reproducibility on consumer hardware.

Three conditions should be compared:

**Baseline A (Sequential hard blocking):** branches execute sequentially, each waiting for the full output of the previous branch before beginning. This establishes the worst-case latency baseline and the best-case accuracy baseline, since sequential execution eliminates false convergence by construction.

**Baseline B (Parallel, no synchronization):** branches execute in parallel with no inter-branch communication. Each branch receives only the original question and its assigned subquestion. This establishes the best-case latency baseline and quantifies the accuracy cost of removing all synchronization.

**TRS:** branches execute in parallel with access to the shared SLP store. The informational bus and value bus are active. The profile layer is not active in this initial experiment, to isolate the contribution of the heat map architecture from the profile layer's history signal, which requires a longitudinal experiment to evaluate properly.

Primary metrics include wall-clock latency, answer accuracy (exact match and F1), and false convergence rate. The **kill criterion** is a false-convergence rate exceeding 15% in the TRS condition, which would invalidate the z-score gate assumption and require architectural revision.

Secondary metrics of interest include value commit timing — the distribution of the interval between value commit and branch termination, which quantifies how much latency the value bus actually saves — and hot-entity overlap between branches, which measures how much cross-branch priming is occurring via the informational bus.

### 8.2 Entropy-Based Recalibration

The fix for Limitation 1 requires implementing a KL-divergence-based drift detector on the per-entity-type heat distributions. We recommend a sliding window estimator that computes the empirical distribution of heat values over the last W writes per entity type and compares it to the running baseline using KL divergence. When divergence exceeds a threshold, the running statistics are reset and the gate enters a warming period during which it applies a more permissive threshold to avoid excessive false negatives during re-estimation. The window size W and divergence threshold are additional empirically tuned hyperparameters whose sensitivity to the rate of domain shift is a key design question.

### 8.3 Value Bus Full Implementation and Evaluation

The current SLP-50 results do not include a live value bus evaluation; the benchmark measures only informational bus MRR. A full value bus implementation requires: instrumenting the reasoning branches to emit confidence-annotated value candidates during generation; implementing the commit-trigger logic; implementing the rollback machinery for tentative commits; and measuring the distribution of value commit timing across a multi-hop benchmark dataset. This measurement will determine whether the early-commit assumption holds empirically — if most values are not determined until the final sentence of a branch's output, the value bus provides limited latency benefit over hard blocking.

### 8.4 Profile Layer Longitudinal Evaluation

The profile layer's maturity curve is a theoretical prediction that requires longitudinal evaluation. A natural experimental design is a simulated multi-session experiment where a held-out set of queries is presented to a TRS system in repeated sessions, with the profile accumulating signal across sessions. MRR and value-prediction accuracy should be measured at each session boundary to trace the maturity curve and validate the staleness detector's ability to recover from induced behavioral shifts.

### 8.5 Scaling to Larger Branch Counts

The write contention analysis suggests that the current spinlock design will not scale beyond approximately four simultaneous branches. A lock-free design using compare-and-swap on heat values, combined with a partitioned store where different entity type codes map to different partitions, would allow higher branch counts without contention. Alternatively, a dedicated aggregator thread that collects branch writes in per-branch queues and applies them to the store in batches could decouple write throughput from branch count at the cost of a small additional latency for heat updates.

---

## 9. TRS as Model-Agnostic Memory Middleware

The preceding sections have characterized TRS as a synchronization layer for parallel reasoning branches executing concurrently within a single inference session. This framing, while accurate, understates the scope of the architecture. TRS is also a persistent memory layer that any LLM can read from and write to, making domain knowledge durable across model swaps, session boundaries, and context window limits. In this section we develop the middleware framing and articulate three problems it solves that are independent of parallelism.

The core argument is a separation that current LLM deployments elide: a language model is stateless compute. Its weights encode general knowledge; its context window encodes session state; neither persists beyond the generation call. The user's domain knowledge — which entities matter to their work, which values recur, which reasoning paths they have already traversed — lives nowhere durable. It must be re-supplied at the start of every session, re-established after every model switch, and increasingly truncated as context windows fill. TRS addresses this by placing a stateful memory layer beneath the model, outside the context window, persistent across calls. The LLM becomes interchangeable stateless inference; the heat map is the durable layer that carries the user's domain.

**Problem 1: Model-swap cold start.** Users increasingly operate across multiple LLM providers within a single working day. A research query goes to one model; a coding task goes to a specialist model; a synthesis task goes to a third. Each model starts cold. The user must re-explain their project, their terminology, their preferences, and the thread of the prior conversation. This re-explanation overhead is not a minor inconvenience — for a user in a specialized technical domain, establishing sufficient context for a useful response can cost dozens of turns that must be repeated with each new model.

TRS eliminates this overhead. The heat map persists outside any model's context window as a standalone data structure. When a user switches from one LLM provider to another

mid-session, the new model's first query issues `preview_query(user_query)`, which retrieves the top-k hot entities from the persistent heat map and injects them into the prompt. The new model begins its first generation with the same pre-warmed entity set that the prior model had accumulated after dozens of turns of collaborative reasoning. From the user's perspective, the new model does not start cold — it starts oriented, with the domain's active entities already salient.

**Problem 2: Context window degradation.** In long sessions, context windows fill. As the session grows, older turns are truncated from the context to make room for newer ones. Established facts that the model referenced freely early in the session — a confirmed architecture decision, a resolved ambiguity, a user preference — may fall off the context window edge and be silently forgotten. The model that was expert in the user's domain at turn 20 becomes progressively less reliable at turn 100. Performance degrades monotonically as session length grows.

TRS inverts this relationship. Retrieval from the heat map is query-specific, not positional. Turn 200 is as sharp as turn 5 because what gets loaded is determined by heat — by the current query's relevance to entities that have accumulated salience over the session — not by recency or position in a context buffer. Moreover, the heat map improves as the session progresses: more turns mean more spreading activation, a more calibrated z-score gate, and a richer profile layer. The system becomes more accurate the longer the session runs, not less. This is the direct inverse of context-window degradation and represents a qualitative behavioral difference from all purely prompt-based session management approaches.

**Problem 3: Cross-session and cross-model consistency.** A user who switches between models across sessions — using one model Monday, a different model Thursday — receives inconsistent behavior. Each model has no memory of the prior session, no knowledge of decisions made or context established. The user must maintain continuity manually, which in practice means they either re-explain extensively or accept that the new model will make different assumptions. For a user engaged in long-running knowledge work across weeks or months, this inconsistency compounds into a significant productivity cost.

With TRS as middleware, all models read from and write to the same persistent heat map. Consistent context, consistent entity salience, consistent value predictions — regardless of which model handles a given query, regardless of when the query is issued. The user's heat map is their durable cognitive workspace, and any model that implements the two-call integration pattern becomes a compatible inference engine for that workspace.

**The infrastructure analogy.** TRS is to LLMs what a database is to application code. Application code is stateless compute: a function call reads inputs, produces outputs, and terminates. Persistence lives in the database, which any code tier can read from and write to. The database is the durable layer; code is interchangeable. LLM providers become interchangeable stateless inference engines; the user's heat map is the durable layer that carries domain knowledge, query history, and behavioral profile. Just as switching from one application server framework to another does not destroy the database, switching from one LLM provider to another does not destroy the heat map.

**Integration pattern.** The integration is minimal and model-agnostic. Any LLM call adds exactly two operations. Before generation, `context = preview_query(user_query)` scans the heat map and returns the top-k hot entities; these are injected into the system prompt as a compact context block. After generation, `process_turn(model_output)` runs the stream watcher over the model's response, updating heat values and committing any value-bus writes. The first call runs in approximately 2 milliseconds for a 150 KB int8 scan. The second call runs asynchronously after the generation completes, adding no latency to the user-facing response. Neither call requires modification to the LLM itself, access to its weights, or knowledge of its internal architecture. Any model accessible via a standard completion API can be integrated.

These two calls are, in essence, the interface definition for TRS as middleware: read before generation, write after generation. The heat map handles spreading activation, z-score gating, LRU eviction, and profile prediction internally. The calling code needs no awareness of these mechanisms. TRS's internal architecture is an implementation detail; the interface is two function calls.

---

## 10. Conclusion

We have presented the Thermal Reasoning Substrate (TRS), a shared compact memory architecture for parallel LLM inference that replaces hard causal synchronization with soft, threshold-gated dependency. TRS consists of three channels: a 128-dimensional int8-quantized entity heat map with spreading activation and z-score gating (informational bus), a typed key-value store with mid-stream commit semantics (value bus), and a personalized behavioral profile that functions as a value predictor (profile layer). Together, these channels convert inter-branch dependencies from binary blocking events into continuous heat-mediated signals, reducing pipeline latency by the duration of post-commit generation in the value-dependency case and to near-zero in the informational-dependency case.

The theoretical foundations of TRS span four decades of cognitive science and computer architecture: Global Workspace Theory's shared broadcast medium, Collins and Loftus's spreading activation, the Tomasulo algorithm's out-of-order execution via a common data bus, Lipasti's value prediction, and the Hearsay-II blackboard architecture's typed hypothesis coordination. TRS is, to our knowledge, the first system to apply this combination of ideas to the problem of inter-branch coordination in parallel LLM reasoning.

On the SLP-50 internal benchmark, TRS achieves MRR 0.940 after stream-replay warming, exceeding a full embedding-scan engine (MRR 0.739) by a factor of 1.27 while maintaining a fixed 150 KB working set versus the engine's O(n) memory footprint. Perfect MRR on infrastructure queries, high performance on task queries, and robust performance on adversarial probes confirm that the z-score gate successfully suppresses low-confidence activations without excessively restricting genuine signals.

We acknowledge five limitations — stationarity assumptions in z-score gating, store saturation risk, rollback machinery complexity, false-convergence risk, and write contention at scale — and propose rigorous validation on MuSiQue and HotpotQA with a 15% false-convergence kill criterion. The most important open question is whether the z-score gate's stationarity

assumption holds in practice across diverse multi-hop reasoning tasks and user populations. If it does, TRS's approach to soft dependency gating offers a practical path to low-overhead parallel reasoning at the edge, enabling 1-3B models to reason cooperatively with the structural sophistication previously available only to much larger models on server hardware.

The combination of GWT-inspired broadcast, spreading activation, and CPU-architecture-style value prediction represents a new design point in the space of parallel reasoning architectures — one oriented toward continuous, quantized, sub-token communication rather than the discrete text-based synchronization that dominates current practice.

---

## References


[1] Baars, B. J. (1988). *A Cognitive Theory of Consciousness*. Cambridge University Press.

[2] Dehaene, S., Changeux, J.-P., & Naccache, L. (2011). The global neuronal workspace model of conscious access: From neuronal architectures to clinical applications. *Experimental Brain Research*, 206(2), 81-101.

[3] Collins, A. M., & Loftus, E. F. (1975). A spreading-activation theory of semantic processing. *Psychological Review*, 82(6), 407-428.

[4] Tomasulo, R. M. (1967). An efficient algorithm for exploiting multiple arithmetic units. *IBM Journal of Research and Development*, 11(1), 25-33.

[5] Lipasti, M. H., Wilkerson, C. B., & Shen, J. P. (1996). Value locality and load value prediction. *Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS VII)*, 138-147.

[6] Erman, L. D., Hayes-Roth, F., Lesser, V. R., & Reddy, D. R. (1980). The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2), 213-253.

[7] Hao, G., Dai, Y., Qin, X., & Yu, S. (2026). Brain-Inspired Graph Multi-Agent Systems for LLM Reasoning. *arXiv:2603.15371*.

[8] Shang, W. (2026). Theater of Mind for LLMs: A Cognitive Architecture Based on Global Workspace Theory. *arXiv:2604.08206*.

[9] Chen, W., et al. (2026). Parallel-Synthesis: Efficient KV Cache Sharing for Parallel Reasoning Branches. *arXiv:2606.14672*.

[10] Park, J., et al. (2025). Native Parallel Reasoner: Topological Scheduling for LLM Inference Graphs. *arXiv:2512.07461*.

[11] Kumar, A., et al. (2025). Blackboard LLM: A Shared Hypothesis Store for Multi-Agent Language Model Systems. *arXiv:2507.01701*.

[12] Wang, H., et al. (2025). SpecBranch: Speculative Multi-Branch Decoding for Large Language Models. *arXiv:2506.01979*.

[13] Sanyal, A. (2026). loci-engine (v1.0.0). GitHub. https://github.com/arkosanyal/loci-engine/releases/tag/v1.0.0

[14] Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D. (2018). Neural Ordinary Differential Equations. *Advances in Neural Information Processing Systems*, 31.

[15] Gu, A., Goel, K., & Ré, C. (2022). Efficiently Modeling Long Sequences with Structured State Spaces. *International Conference on Learning Representations*.

[16] Kidger, P., Morrill, J., Foster, J., & Lyons, T. (2020). Neural Controlled Differential Equations for Irregular Time Series. *Advances in Neural Information Processing Systems*, 33.


---

## Appendix A: Population-Scale Emergence and Socioeconomic Stratification

The analysis in the main paper treats TRS as a single-user system: one heat map, one profile, one session history. This framing is the right unit of deployment — TRS's privacy properties and edge-deployment model depend on data locality. But it understates the architectural potential of the profile layer when considered at scale. This appendix develops the case for what becomes possible when heat map profiles aggregate across users, and identifies both the opportunity and the constraints that govern it.

**The compounding claim.** A single user's profile predicts that user's next queries, bounded by that user's history. The predictions are useful — they reduce stall, pre-warm entities, and improve retrieval quality — but they are idiosyncratic. They reflect personal habits, personal vocabulary, and a single reasoning trajectory through one domain. The predictions do not generalize.

But as profiles aggregate across users, a different class of pattern becomes statistically visible: cross-user invariants. These are patterns that hold regardless of individual quirks — query trajectories that converge because they reflect the problem space, not because different users share habits. A developer debugging a distributed system will, regardless of personal style, eventually query for network topology, then service dependencies, then failure modes. The sequence is driven by the logical structure of the problem, not by personal preference. Cross-user invariants encode this logical structure. They are stronger predictions than individual-profile predictions because they are grounded in the problem domain, not in one person's history.

**Three-phase emergence model.** The transition from individual to cross-user prediction follows a three-phase model.

In the **individual phase**, the profile predicts for one user within their domain. Accuracy is bounded by history length: early sessions have sparse data and low accuracy; mature sessions accumulate sufficient signal to predict reliably for recurring query types. The system learns this user's query vocabulary, entity preferences, and reasoning patterns. All of this is fully realizable without any cross-user data sharing.

In the **cohort phase**, profiles aggregated across users in similar contexts reveal domain-level patterns. Consider the specific context of a developer building distributed AI systems under resource constraints — a context that can be inferred from query patterns without requiring the user to self-report it. Such a developer's query trajectory overlaps predictably with other developers in similar contexts: the queries about memory footprints, about edge inference tradeoffs, about open-source model selection, about bandwidth constraints. These overlaps are not coincidental and not the result of shared personal habits. They are the result of the problem space converging. A developer in this context who has asked about memory efficiency is highly likely to ask next about quantization, about pruning, or about context window management — not because most developers ask these things, but because developers in this context almost always ask these things. Cohort-level priors make this prediction available from session one of a new user whose context matches the cohort, before their individual profile has accumulated

any signal.

In the **population phase**, cultural and socioeconomic stratification enables anticipatory context loading at a level of specificity that individual and cohort phases cannot reach. A user whose query patterns consistently include "lightweight," "edge," "free tier," "open source," and "bandwidth" is signaling, through their vocabulary, a resource constraint that shapes every architectural decision they make. A system that infers this context can pre-warm entities around cost-efficient solutions before the user explicitly asks — not because it knows their financial situation, but because their query vocabulary encodes the constraints they are operating under. A user in a regulatory environment (GDPR, data residency requirements, sovereignty constraints) queries differently than a user without those constraints; the vocabulary of compliance — "data residency," "on-premises," "sovereignty," "audit log" — is a reliable signal of the regulatory context, and that context predicts an entire class of architectural queries that follow. Population-phase prediction operates at the level of inferred context, not just observed history.

**Superlinear improvement.** Each new user profile does not add one data point to the aggregate — it potentially unlocks a new cluster that retroactively benefits all prior users in that cluster. A new user whose profile, once mature, is identified as belonging to an existing cohort immediately inherits that cohort's priors. Equally important, a new user whose profile does not match any existing cohort may, once mature, define a new cohort that retroactively improves predictions for prior users whose individual-phase profiles were underfit because their cohort was too small to be statistically visible. This mirrors the qualitative phase transition observed in language model scaling: a billion tokens does not give a billion times better predictions, it gives qualitatively new capabilities because sub-threshold patterns become statistically visible. The heat map aggregate has the same property. Prediction quality improves non-linearly with the number of profiles because cluster definition improves, cross-user invariants become sharper, and the cohort prior becomes more informative.

**The socioeconomic signal.** Query patterns encode economic reality with a fidelity that users rarely realize they are providing. Users in bandwidth-constrained environments make systematically different architectural choices than users with abundant compute: they query more about compression, about quantization, about batching efficiency, about local inference. Users building for constrained hardware make different tradeoffs than users building for cloud deployment. These differences are not noise — they are signal about the user's operating context that makes their future queries more predictable. A population-aware TRS can stratify profiles by inferred context and use cohort priors to improve individual predictions without requiring users to self-report their context. The inference is implicit and behavioral, derived entirely from what the user asks and how they ask it.

**Privacy constraint and mitigation.** Population-phase aggregation requires sharing behavioral signals across users. Raw profile sharing is a non-starter. A heat map profile contains enough information to infer a user's domain, role, organization type, and operational constraints — a privacy exposure that no deployment should accept.

The viable path is federated learning. Each device trains on local profile data. What is shared is not the profile itself but gradient updates: the direction and magnitude of parameter changes

derived from local training, not the training data itself. A shared cohort model aggregates these gradients, improving the population-phase prior for all participants. Differential privacy guarantees, applied at the gradient level, bound each individual's contribution to the cohort model: the mathematical guarantee ensures that no individual user's queries can be reconstructed from the published gradient updates, even by an adversary with auxiliary information. This is an active research area with mature implementations; the constraint is engineering effort, not fundamental feasibility.

The individual and cohort phases are fully realizable without any cross-user data sharing. Population-phase benefits require federated infrastructure, but individual-phase value — the reduction in cold-start overhead, context window degradation, and cross-session inconsistency described in Section 9 — is available to any single-user deployment. The population phase is an optional architectural extension, not a prerequisite.

**Implication for edge deployment.** The population phase makes the edge deployment case stronger over time. A 1-3B model running on-device with a locally-trained cohort prior can outperform a 7B cloud model on domain-specific tasks, because the cohort prior encodes domain knowledge that the 7B model carries in weights but cannot apply selectively. The 7B model knows generally about distributed systems; the edge model with a cohort prior knows specifically about the reasoning patterns of developers in resource-constrained distributed AI contexts. The edge model's 150 KB heat map, trained on real user behavior in that domain, encodes a form of domain expertise that parameter count alone cannot replicate. As more users in similar contexts contribute to cohort priors via federated learning, this advantage compounds. The population phase does not require larger models — it requires better priors, and better priors come from behavioral aggregation, not from additional weights.

---

## Appendix B: Dual-Stream Memory Schema — isymprev and qsymprev

The TRS informational and value buses, described abstractly in the main paper, are concretely implemented as two parallel storage streams with distinct schemas, retrieval semantics, and heat propagation rules.

### B.1 isymprev — Informational Stream

isymprev stores narrative, relational, and contextual entity knowledge. It is the heat map's ground layer: the surface on which spreading activation propagates.

```
CREATE TABLE isymprev (
entity TEXT PRIMARY KEY,
gist TEXT, -- one-line narrative summary
heat REAL DEFAULT 1.0, -- 0.0–3.0, decays 0.95x/day
confidence REAL DEFAULT 1.0,
uses INTEGER DEFAULT 0,
last_used REAL, -- unix timestamp
expanded TEXT -- full description if available
);
```

Heat activation is continuous and neighbor-propagating:

- Direct access: heat += 1.0, resets decay clock
- 1-hop graph neighbors: heat += 0.5
- 2-hop neighbors: heat += 0.25
- 3-hop neighbors: heat += 0.125
- Passive decay: heat *= 0.95 per day

Storage tier is derived from heat:

- HOT (heat > 1.5): embedding loaded as CUDA tensor, sub-millisecond retrieval
- WARM (heat > 0.5): numpy array in RAM, ~5ms retrieval
- COLD (heat ≤ 0.5): NVMe/SQLite, loaded on demand, ~50ms

**isymprev is not suitable for precise factual queries.** It encodes what is contextually relevant, not what is quantitatively true.

### B.2 qsymprev — Quantifiable Stream

qsymprev stores exact, typed, timestamped values attached to isymprev entities. It is the value bus's persistent layer.

```
CREATE TABLE qsymprev (
id INTEGER PRIMARY KEY AUTOINCREMENT,
entity TEXT NOT NULL, -- foreign key to isymprev.entity
key TEXT NOT NULL, -- "port", "MRR", "serial", "height"
value TEXT NOT NULL, -- exact value: "8766", "0.933", "6ft"
unit TEXT, -- "tcp", "score", "feet", "date"
ts REAL, -- when this fact was recorded
heat REAL DEFAULT 1.0,-- inherits from parent isymprev entity
confidence REAL DEFAULT 1.0,
source TEXT -- provenance: "measured", "paper", "user"
);
CREATE INDEX idx_qsymprev_entity ON qsymprev(entity);
CREATE INDEX idx_qsymprev_key ON qsymprev(key);
```

qsymprev supports two retrieval modes:

**Mode A — Entity-driven (both streams):**

isymprev activates relevant entities via heat → qsymprev delivers their precise values.

Use case: conversational context where narrative activates then facts are needed.

**Mode B — Direct key lookup (qsymprev only):**

Structured query by key pattern, bypassing isymprev entirely.

Use case: exact fact retrieval where no narrative context is required.

Example — lottery ticket lookup:

```
Query: "July 14 lottery tickets"
→ Mode B: SELECT value FROM qsymprev
WHERE entity LIKE '%14J%' AND key='lottery_ticket'
→ Returns: 241410, 241411
Encoding: 14JLLT241410 = day(14) month(J) type(LL=lottery) ticket(T) serial(241410)
```

The LLM retrieves exact serials with zero hallucination risk — the value is either in qsymprev or the system returns "not found."

### B.3 Predictive Context Loading

Both streams support anticipatory retrieval triggered at keystroke cadence:

```
User types partial input → debounced entity extraction (150ms)
→ isymprev heat activation for matching entities
→ qsymprev pre-fetch for hot entities
→ context assembled before message submits
```

Over sessions, access patterns populate loci_predictions:

```
loci_predictions: query_fp → next_entities, frequency, confidence
```

The system transitions from reactive (respond to keystrokes) to predictive (pre-warm before the topic appears). With sufficient sessions, "dad" pre-activates fishing context before "fish" is typed, because the co-occurrence pattern has been observed across hundreds of sessions.

### B.4 Confidence Gate

Both streams feed a confidence gate before context is injected:

```
correction fires only if:
checker.confidence > isymprev_entity_heat

High-heat entity (well-established) → overrides LLM at moderate mismatch
Low-heat entity (uncertain) → only fires on severe mismatch (stub/hallucination)
```

This prevents over-correction on uncertain knowledge while ensuring high-confidence ground truth always wins.